Juha Laurila / Databases

Getting started with databases

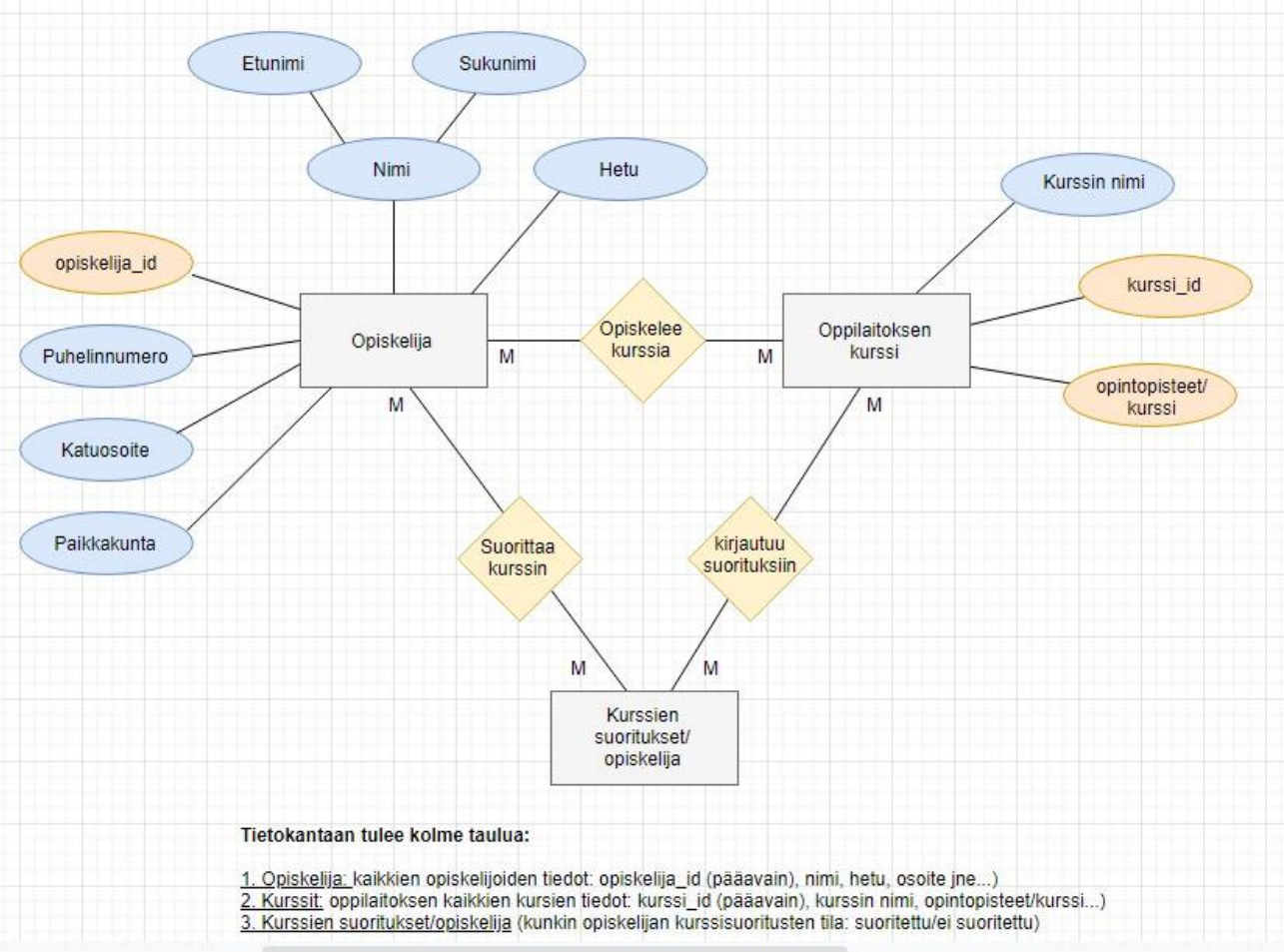
Many years ago, I heard first time about some kind of database, MySql. During my job history, there have been many times I have been in any project that has been used databases. In principle, I am quite familiar with situations and necessities to use databases generally. However, the actual creating and implementing of databases have been perfectly unknown to me until now.

Task 1: Planning the student register

First of all, I began to think how I would do a database like this in principle. Before to trying tasks at all, I watched all the videos located in Moodle site of database course. After watching the videos I was quite familiar with best methods to design databases. The first step would be to think relationships of things and implement ER diagram for it. First, as mentioned in instructions of the task, I noticed that there have to be three entitities in ER diagram. According the planning principles, all the data should be saved only once in the database. So, One of the entities is a student with all the data of a student like name, address and so on. Secondly, I needed the entity for the courses of school, including school subjects, subject_id and credits/certain subject, for example. Third entity should be dedicated for data of the completions of courses of students. This entity will be a compound table where is completion_id as a primary key and student_id and courses_id as a foreign keys. In addition, in this table is also a status of completion of a course - true or false, for example. Now, in my opinion, I have basis principles for a database of student register ready. So, all the queries needed to use this database are possible and all the data of the database have been saved only once.

Eventually, I spent a few hours to complete this task. But of course, getting started with videos and other material mentioned above took much more time. I think, with this task I a have came familiar with principles and importance of designing ER diagrams to ensure myself that the final database will be well-functioning and has be donein

accordance with good design principles.



Task 2: Online store database Design

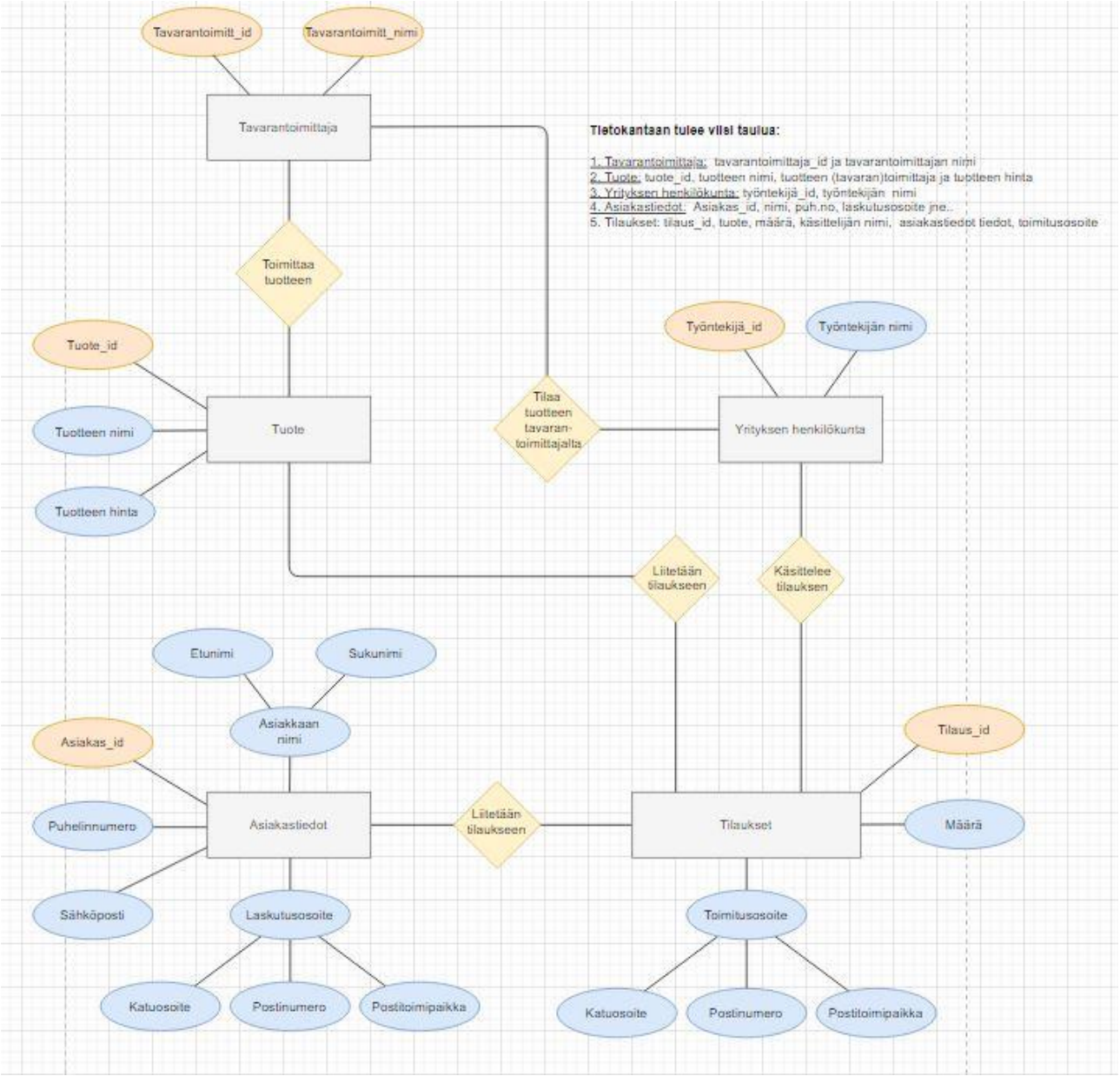
The second task was a little bit more complex and consisted of more entities. However, it was easier to go forward with online store database task because I studied er diagrams with first task.
First operation of all was to draw entities and all the things which has joined directly to them, such as id:s and names. Secondly, I thought relationships and dependencies between entities. Some of the entities contains only basic data such as Asiakastiedot and Tuotetiedot. Tilaukset entity was different instead; it was the so-called compound entity, which contains data from two or more basic entity. In many case, if the relationship between two entities are many-to-many type, it is better to add extra entity between them as a compound entity. After that it will be much more easier to plan database according instructions of database planning principles, and in addition, to make database queries.

One of the most important issue also with this database was the integrity of data. All the data should be written only one time in the database.

In the ER diagram below, I tried to think quite a realistic situation of function of online store. At first, when a customer begin to make an order in the online store, all

necessary information related to the customer is written to the entity Asiakastiedot. Secondly, when the customer decided a product to be buyed, all the data of product is located in Tuotteet database. At last, when the customer has written all the data needed for making order, its time to press the send an order -button.

At that moment the Tilaukset entity will receive data from Asiakastiedot and Tuotteet entities. On the other words, data is saved to the Tilaukset database. In the reality of the online store, somebody of the staff of the online store begin to handle the order. One of the operations is to order right amount of right products from Tavarantoimittaja and handle they to be sent to the customer.
Finally there will be datas of customer, data of product with its Tavarantoimittaja, price, amount of products and data of the person who handled the order. A certain order in the Tilaukset entity has been identified with Tilaukset_id. In this way it is as an so called compound table completed with foreign keys by Yrityksen henkilökunta, Tuote, Asiakastiedot and Tavarantoimittaja-entities (tables).



Some basic principles with databases

According to many different sources I read in addition to this course material, there are some certain very important principles with design of databases. At first, before implementing final database tables, it is important to do a graphic database schema. Before the schema scetching it is extremely important important to think and understand all the big entities and relationships between them. For example, if it would be question of planning online store, all the entities, different parties of online buying event and relationships between them, have to be aware. At the beginning of that, there would be three kind of relationships, one to many, many to many and one to one. Usually in databases, the relationships are one to many type. If there are many to many types, they have to change one to many type relationship implementing third table between them.
All the tables have to include one column for primary key. In the case of database with

multiple tables, depending of relationships of tables, there are always need also foreign keys, which refers to primary key in other table. So there is possible to do complex queries from multible tables where are dependies in keys. In additions, when data of database have to change or delete within time period, well implemented relationships works well also in such a situations, so one of the main principles, the integrity of database maintains.
One of the most important principles is this: one data only to one place. In addition, every field in database table was dedicated only for one data. In addition, every column was dedicated to certain data type. Third, all the specific data stored in tables, which can be get derived by queries from tables, is useless, becaucse it causes the repetations of data which is opposite of the basic principles of database design.

MariaDB installation

Before this course, I had Docker environment in my local machine yet, so there was only the installation of MariaDB to the container according to instructions. The installation procedure was quite easy with the guide and in addition, I had used Docker in earlier courses.
Soon, I have MariaDB installed and in use, and it was time to try out the using of the database. So, I went to “Tietokantaharjoitus 1 - Database exercise1”.

Task3. Tietokantaharjoitus 1 - Database exercise1

At first, there was a little challenge to get started, with a losen password. Finally I found it in materials and began to test the database. Quite quickly I learned the routines of logging in to the database and basic commands to use it.

In the image below, there are some basic sql queries to creating a new table with certain column, adding new column afterwards and describing the construction of a certain table. In addition, the excercise learned how to add PRIMARY KEY determination and auto incrementation function to the id number.

After this excercise, I had clear understanding how to use basic functions and sql queries in terminal using MariaDB database in practise. Before this task I watced all the videos related to sql queries and thats why the things were quite clear for me in principle.

```
MariaDB [test_db]> CREATE TABLE pet (name VARCHAR(20), owner VARCHAR(20)) ENGINE=InnoDB
DEFAULT CHARSET=latin1;
Query OK, 0 rows affected (0.119 sec)

MariaDB [test_db]> SHOW TABLES;
+--------------------+
| Tables_in_test_db |
+--------------------+
| pet                |
+--------------------+
1 row in set (0.002 sec)

MariaDB [test_db]> ALTER TABLE pet ADD COLUMN id INT NOT NULL PRIMARY KEY AUTO_INCREMENT;
Query OK, 0 rows affected (0.075 sec)
Records: 0  Duplicates: 0  Warnings: 0

MariaDB [test_db]> SHOW TABLES;
+--------------------+
| Tables_in_test_db |
+--------------------+
| pet                |
+--------------------+
1 row in set (0.002 sec)

MariaDB [test_db]> DESCRIBE pet;
+-------+-------------+------+-----+---------+----------------+
| Field | Type        | Null | Key | Default | Extra          |
+-------+-------------+------+-----+---------+----------------+
| name  | varchar(20) | YES  |     | NULL    |                |
| owner | varchar(20) | YES  |     | NULL    |                |
| id    | int(11)     | NO   | PRI | NULL    | auto_increment |
+-------+-------------+------+-----+---------+----------------+
3 rows in set (0.022 sec)
```

Task4. Tietokannan kyselyt - harjoitus 4

In this excercise I practiced more of use of basic sql queries. At first, I had to create simple tables with certain columns. Following tasks learned to me more complex queries, such as to create a duplicate copy of countries table including structure and data by name dup_countries. Next i had to create tables with constraint NULL, setting limits to query and many many other basic operation.

```
MariaDB [sql_kyselyt]> CREATE TABLE jobs (
    -> jobs_id VARCHAR(10) NOT NULL DEFAULT '',
    -> jobs_title VARCHAR(35) NOT NULL,
    -> min_salary DECIMAL(6,0) DEFAULT NULL,
    -> max_salary DECIMAL(6,0) DEFAULT NULL,
    -> CHECK (max_salary<=25000),
    -> PRIMARY KEY (jobs_id)
    -> ) ENGINE=MyISAM DEFAULT CHARSET=latin1;
Query OK, 0 rows affected (0.028 sec)
```

```
MariaDB [sql_kyselyt]> DESCRIBE jobs;
+------------+---------------+------+-----+---------+-------+
| Field      | Type          | Null | Key | Default | Extra |
+------------+---------------+------+-----+---------+-------+
| jobs_id    | varchar(10)   | NO   | PRI |         |       |
| jobs_title | varchar(35)   | NO   |     | NULL    |       |
| min_salary | decimal(6,0)  | YES  |     | NULL    |       |
| max_salary | decimal(6,0)  | YES  |     | NULL    |       |
+------------+---------------+------+-----+---------+-------+
4 rows in set (0.046 sec)

MariaDB [sql_kyselyt]> |
```

Finally, after completing the task, I had knowledge of basics of queries made from a single table. In my opinion, the most important things I learned was creating a table with different determinations, modifying a table by adding columns, updating datas and so on, queries by different order of data (ASC/DESC), limits and joining different calculation operations to queries, min/max/avg values etc. I noticed, that it is very easy to get effectively different data using different SELECT and also UPDATE queries.

```
MariaDB [sql_kyselyt]> SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME, SALARY
    -> FROM employees
    -> ORDER BY SALARY ASC;
+-------------+-------------+-------------+----------+
| EMPLOYEE_ID | FIRST_NAME  | LAST_NAME   | SALARY   |
+-------------+-------------+-------------+----------+
|         132 | TJ          | Olson       |  2100.00 |
|         136 | Hazel       | Philtanker  |  2200.00 |
|         128 | Steven      | Markle      |  2200.00 |
|         135 | Ki          | Gee         |  2400.00 |
|         127 | James       | Landry      |  2400.00 |
|         119 | Karen       | Colmenares  |  2500.00 |
|         191 | Randall     | Perkins     |  2500.00 |
|         140 | Joshua      | Patel       |  2500.00 |
|         182 | Martha      | Sullivan    |  2500.00 |
|         131 | James       | Marlow      |  2500.00 |
|         144 | Peter       | Vargas      |  2500.00 |
|         143 | Randall     | Matos       |  2600.00 |
|         199 | Douglas     | Grant       |  2600.00 |
|         118 | Guy         | Himuro      |  2600.00 |
|         198 | Donald      | OConnell    |  2600.00 |
|         126 | Irene       | Mikkilineni |  2700.00 |
|         139 | John        | Seo         |  2700.00 |
|         117 | Sigal       | Tobias      |  2800.00 |
|         195 | Vance       | Jones       |  2800.00 |
|         130 | Mozhe       | Atkinson    |  2800.00 |
|         183 | Girard      | Geoni       |  2800.00 |
|         134 | Michael     | Rogers      |  2900.00 |
|         116 | Shelli      | Baida       |  2900.00 |
|         190 | Timothy     | Gates       |  2900.00 |
|         197 | Kevin       | Feeney      |  3000.00 |
```

```
MariaDB [sql_kyselyt]> SELECT SUM(SALARY)
    -> FROM employees;
+-------------+
| SUM(SALARY) |
+-------------+
|   691400.00 |
+-------------+
1 row in set (0.001 sec)

MariaDB [sql_kyselyt]> |
```

```
MariaDB [sql_kyselyt]> UPDATE employees
    -> SET SALARY = 8000
    -> WHERE EMPLOYEE_ID=105 AND SALARY<5000;
Query OK, 1 row affected (0.021 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [sql_kyselyt]> SELECT * FROM employees;
```

```
MariaDB [sql_kyselyt]> SELECT * FROM employees
    -> WHERE EMPLOYEE_ID = 105;
+-------------+------------+-----------+---------+--------------+-------------+---------+---------+----------------+------------+---------------+
| EMPLOYEE_ID | FIRST_NAME | LAST_NAME | EMAIL   | PHONE_NUMBER | HIRE_DATE   | JOB_ID  | SALARY  | COMMISSION_PCT | MANAGER_ID | DEPARTMENT_ID |
+-------------+------------+-----------+---------+--------------+-------------+---------+---------+----------------+------------+---------------+
|         105 | David      | Austin    | DAUSTIN | 590.423.4569 | 1987-06-22  | IT_PROG | 8000.00 |           0.00 |        103 |            60 |
+-------------+------------+-----------+---------+--------------+-------------+---------+---------+----------------+------------+---------------+
1 row in set (0.001 sec)

MariaDB [sql_kyselyt]> |
```

Task 5. Company

Next it was time to go to operate with much more complex database called company. First of all, I created a new database and begin to creat all the tables needed in the database. I created all the tables according to video example: employee, branch, client, works_with and brand_supplier tables.

```
+-------------------+
| Tables_in_company |
+-------------------+
| branch            |
| branch_supplier   |
| client            |
| employee          |
| works_with        |
+-------------------+
5 rows in set (0.001 sec)

MariaDB [company]> |
```

Secondly I had to insert all the data to just created tables by commands explained in company_sql file. This task was relatively straightforward operation, by proceduce like this two ones:

```
MariaDB [company]> INSERT INTO employee VALUES(100, 'David', 'Wallace', '1967-11-17', 'M', 250000, NULL, NULL);
Query OK, 1 row affected (0.012 sec)
```

```
MariaDB [company]> INSERT INTO branch VALUES(1, 'Corporate', 100, '2006-02-09');
Query OK, 1 row affected (0.007 sec)
```

There were many insertion and update operations to different tables in the task, like mentioned above.

After that all was ready to begin to insert queries to the database. Almost all of the queries were SELECT type. Select queries are extremely effective operations to get many kind of data from databases. At first there were simple queries like this:

SELECT *
FROM clients;

SELECT *
from employee
ORDER BY sex, name;

```
SELECT first_name AS forename, employee.last_name AS surname
FROM employee;
```

A little bit more complex queries contained different functions, such as AVG, COUNT and SUM, which are very useful if its need to calculate somethings:

```
SELECT AVG(salary)
FROM employee;
```

```
SELECT SUM(salary)
FROM employee;
```

```
SELECT COUNT(sex), sex
FROM employee
GROUP BY sex
```

Wildcards using LIKE determination are effective to find a certain characters from database:

```
SELECT *
FROM branch_supplier
WHERE supplier_name LIKE '% Label%';
```

```
SELECT *
FROM employee
WHERE birth_day LIKE '_____10%';
```

With UNION command it is easy to collect results from many table in one list.

A JOIN clause is used to combine rows from two or more tables, based on a related column between them.

This is two examples of them:

```
SELECT client.client_name AS Non-Employee_Entities, client.branch_id AS Branch_ID
FROM client
UNION
SELECT branch_supplier.supplier_name, branch_supplier.branch_id
FROM branch_supplier;
```

```
SELECT employee.emp_id, employee.first_name, branch.branch_name
FROM employee
JOIN branch – LEFT JOIN, RIGHT JOIN
ON employee.emp_id = branch.mgr_id;
```

In nested queries, a query is written inside a query. The result of inner query is used in execution of outer query:

```
SELECT client.client_id, client.client_name
FROM client
WHERE client.branch_id = (SELECT branch.branch_id
FROM branch
WHERE branch.mgr_id = 102);
```

Finally, I learned a way to add certain triggers to queries. A trigger is a stored procedure in database which automatically invokes whenever a special event in the database occurs.

After completing this task, I suppose I had learned all the most necessary things related to basics of sql queries. Of course, there were also task “more demanding queries”. At first, I tried to solve these challences self, but then I noticed there was answers also in moodle; and, according to my teacher, these queries are rarely used. However, I studeid them and noticed, that queries will be possible to do very specific; even though using them is not so easy any more.